

CVA6 RVH HLVX-to-HLV Execute-Only Disclosure Vulnerability Report

Affected version

Issue package: ../../issues/CV-1

This package was validated on a local CVA6 checkout at revision v5.3.0-168-gcd4c8b17, confirmed at commit cd4c8b17c102, using the cv64a6_imafdch_sv39 Hypervisor configuration. No fix revision has been identified yet.

Background

The issue package states the architectural bug: after a legal HLVX.WU access to an execute-only G-stage page, a later ordinary HLV.WU to the same G-stage page can read through instead of raising a load guest-page fault.

The security PoC uses the same bug in a trusted monitor helper flow, where an internal instruction-fetch helper and a caller-visible data-read helper operate on the same execute-only G-stage page.

Vulnerability type and CWE

Vulnerability type. G-stage permission revalidation failure across a trusted monitor helper boundary.

CWE. Primary: CWE-284 Improper Access Control. Secondary: CWE-200 Exposure of Sensitive Information and CWE-693 Protection Mechanism Failure.

Core architectural defect

The minimal architectural issue is:

```
hlvx.wu x6, (x5)
hlv.wu  x6, (x5)
```

The G-stage leaf PTE is execute-only:

```
- V = 1
- X = 1
- U = 1
- A = 1
- R = 0
```

mstatus.MXR is kept clear. Under those conditions, HLVX.WU may use execute permission and succeed, but the later ordinary HLV.WU is a data read and must raise a load guest-page fault. The issue package shows that CVA6 reaches the PoC's read-through path instead.

The direct-HLV control in the issue package shows that a plain HLV.WU without a preceding HLVX.WU faults as expected. The bug is therefore the permission state reused after the legal HLVX access, not a generic ability for HLV to read execute-only G-stage pages.

The relevant RTL separates ordinary hypervisor loads from HLVX loads:

```
case (lsu_ctrl.operation)
```

```

HLV_B, HLV_BU, HLV_H, HLV_HU, HLV_W,
HSV_B, HSV_H, HSV_W, HLV_WU, HLV_D, HSV_D: begin
    hs_ld_st_inst = 1'b1;
end
HLVX_WU, HLVX_HU: begin
    hs_ld_st_inst = 1'b1;
    hlvx_inst     = 1'b1;
end
default: ;
endcase

```

On the PTW miss path, the DTLB update permission check is explicitly `hlvx_inst_i`-aware. A load can be accepted through execute permission when `hlvx_inst_i` is set:

```

if (
    (pte.a && ((pte.r && !hlvx_inst_i) ||
        (pte.x && (mxr_i || hlvx_inst_i ||
            (ptw_stage_q == S_STAGE && vmxr_i &&
                ld_st_v_i && CVA6Cfg.RVH)))))
    && (!lsu_is_store_i || (pte.w && pte.d) ||
        (ptw_stage_q == G_INTERMED_STAGE && CVA6Cfg.RVH))
) begin
    if ((CVA6Cfg.RVH &&
        ((ptw_stage_q == G_FINAL_STAGE) || !en_ld_st_g_translation_i)) ||
        !CVA6Cfg.RVH)
        shared_tlb_update_valid = 1'b1;
    end else begin
        state_d = PROPAGATE_ERROR;
        if (CVA6Cfg.RVH) ptw_stage_d = ptw_stage_q;
    end
end

```

The later G-stage load-hit path is much narrower: the G-stage access error is derived from the cached guest PTE's U bit, and the load guest-page-fault decision consumes only that reduced signal:

```

d_g_st_access_err = en_ld_st_g_translation_i && !dtlb_gpte.q.u;

if (dtlb_hit_q && lsu_req_q) begin
    lsu_valid_o = 1'b1;

    if (lsu_is_store_q) begin
        if (CVA6Cfg.RVH && en_ld_st_g_translation_i &&
            (!dtlb_gpte.q.w || d_g_st_access_err || !dtlb_gpte.q.d)) begin
            lsu_exception_o.cause = riscv::STORE_GUEST_PAGE_FAULT;
            lsu_exception_o.valid = 1'b1;
        end
    end else begin
        if (CVA6Cfg.RVH && d_g_st_access_err) begin
            lsu_exception_o.cause = riscv::LOAD_GUEST_PAGE_FAULT;
            lsu_exception_o.valid = 1'b1;
        end
    end
end

```

On this hit path, a first HLVX miss can populate translation state using execute permission, while a later ordinary HLV hit is not visibly rechecked against the current operation's G-stage read-permission requirement. The PoC exercises that path: direct HLV faults, but HLVX followed by HLV reads through.

Security relevance

The security setting is a trusted HS/M-mode monitor with two helper paths:

1. an internal instruction-fetch or instruction-emulation helper uses HLVX.WU on an execute-only guest page;
2. a caller-visible data-read helper later uses ordinary HLV.WU on the same G-stage page and is expected to fault because the page is not readable.

The scenario uses trusted monitor/helper code. The caller-visible data-read helper relies on G-stage R=0 and MXR=0 to prevent protected execute-only contents from being copied into a caller-visible buffer.

In a vulnerable CVA6 configuration, a legal internal HLVX.WU can populate the translation state, and the later caller-visible HLV.WU can read data from the same execute-only page instead of faulting. This can disclose code-page literals, embedded policy constants, protected firmware words, JIT-generated code bytes, or other contents that the monitor intended to keep executable but not readable through its data-copy path.

Security PoC

Assumptions

Trusted HS/M-mode code uses G-stage execute-only mappings as a boundary between an internal fetch/emulation path and a caller-visible data-read path. The monitor expects ordinary HLV.WU to fault on R=0 pages when MXR=0.

PoC setup

The PoC maps one guest page at `GVA_PAGE = 0x40000000` with a G-stage leaf PTE containing `V|X|U|A` and no R bit. The page contains:

- `FETCH_WORD = 0x00000013` at `GVA_FETCH = 0x40000000`;
- `SECRET_WORD = 0x5ec01231` at `GVA_SECRET = 0x40000080`;
- an initially zero `attacker_buffer` outside the protected page.

The monitor-internal path performs:

```
li    t0, GVA_FETCH
hlvx.wu x6, (x5)
```

That value is consumed internally and is not copied to `attacker_buffer`.

The caller-visible data helper then performs:

```
li    t0, GVA_SECRET
hlv.wu x6, (x5)
```

If the ordinary HLV incorrectly returns `SECRET_WORD`, the PoC copies it into `attacker_buffer` and exits with code 31.

Security result

- The execute-only page is first touched through a legal monitor-internal HLVX.WU.
- The later caller-visible HLV.WU targets a different offset on the same execute-only G-stage page.
- CVA6 reads `SECRET_WORD` instead of taking the expected load guest-page fault.
- The PoC stores the leaked word into `attacker_buffer` and exits through the `leak_secret_to_attacker` path.

CVA6 evidence

The evidence log is `poc/poc.cva6-h.log`:

```
poc.elf *** FAILED *** (tohost = 31) after 2913 cycles
*** [rvfi_tracer] INFO: Simulation terminated after      2901 cycles!
```

tohost = 31 is the PoC's leak_secret_to_attacker path. That path is reached only after the ordinary HLV.WU reads SECRET_WORD and stores it to attacker_buffer.

Expected result on a compliant core

The HLVX.WU at GVA_FETCH succeeds. The later ordinary HLV.WU at GVA_SECRET raises a load guest-page fault with mtval = GVA_SECRET. The PoC then confirms that attacker_buffer is still zero and exits with code 0.

Observed result on CVA6

CVA6 exits with tohost = 31, meaning the ordinary HLV data-read helper read the protected SECRET_WORD from the execute-only G-stage page and copied it into attacker_buffer.

Impact

- Primary: Confidentiality. Protected execute-only guest-page contents can be disclosed through a monitor data-read helper after an internal HLVX access.
- Secondary: Integrity. If the monitor treats a successful HLV data read as evidence that a page is data-readable or authorized for copying, it can make an incorrect policy decision.

Affected version status

Confirmed affected in a local run of CVA6 revision v5.3.0-168-gcd4c8b17, confirmed at commit cd4c8b17c102, using the cv64a6_imafdch_sv39 Hypervisor configuration.

Fixed: none identified yet.

Fix direction

For G-stage DTLB hits, CVA6 should revalidate the current access type against the cached G-stage PTE permissions. A current ordinary HLV load must require G-stage read permission unless MXR permits execute-only reads. Permission state created by a prior HLVX access must not allow later ordinary HLV reads to bypass R=0.

One repair direction is to derive the G-stage load guest-page-fault condition from the current LSU operation as well as from the cached guest PTE. In particular, an ordinary HLV hit should require dtlb_gpte_q.r, while an HLVX hit may use dtlb_gpte_q.x as the permission source. The direct-HLV control should remain part of the regression set together with the HLVX -> HLV sequence.